CSB-01 · REAL-TIME DETECTION OF CROSS-CHAIN BRIDGE EXPLOITS

BridgeWitness

A page-by-page guide to the prototype, in plain English.

Signatures can be stolen. Conservation can't.

4 ideas · 3 machine-learning models · 6 algorithms

Real data from the Nomad and Ronin hacks · a live bridge feed

October 2026

The prototype from A to Z

The problem

A **bridge** lets you move coins between two blockchains. It locks your coins on chain A (a **deposit**) and pays out the same value on chain B (a **payout**). Bridges hold huge sums, so they are the biggest target in crypto: about $2 billion was stolen from bridges in 2022 (Chainalysis estimate).

Two hacks show why simple alerts fail. At **Ronin** (March 2022) attackers stole validator keys, so every signature was genuine and signature checks saw nothing. At **Nomad** (August 2022) a bug let anyone withdraw money that was never deposited; one person found it, then hundreds copied the same transaction.

Our answer in one sentence

Signatures can be stolen. Conservation can't. Coins paid out must never exceed coins locked. BridgeWitness checks that every payout has a deposit behind it, and only says **EXPLOIT** when that check breaks. Everything else (statistics, machine learning) can only say **SUSPICIOUS** and help explain.

What we measured (on real data)

384 / 384
real attack payouts labelled exploit (Nomad 382, Ronin 2)

0 of 27,312
honest payouts wrongly accused

$159.6M
Nomad money paid out with no deposit

How it works, in five steps

1. **Read** deposits and payouts, replayed from real hacks or read live.
2. **Pair** each payout with one deposit by amount, person and timing.
3. **Measure** spread, volume, signers and size against normal days.
4. **Decide**: EXPLOIT only if the deposit rule breaks.
5. **Explain**: attack type, top reasons, the closest past hack, and a check anyone can repeat.

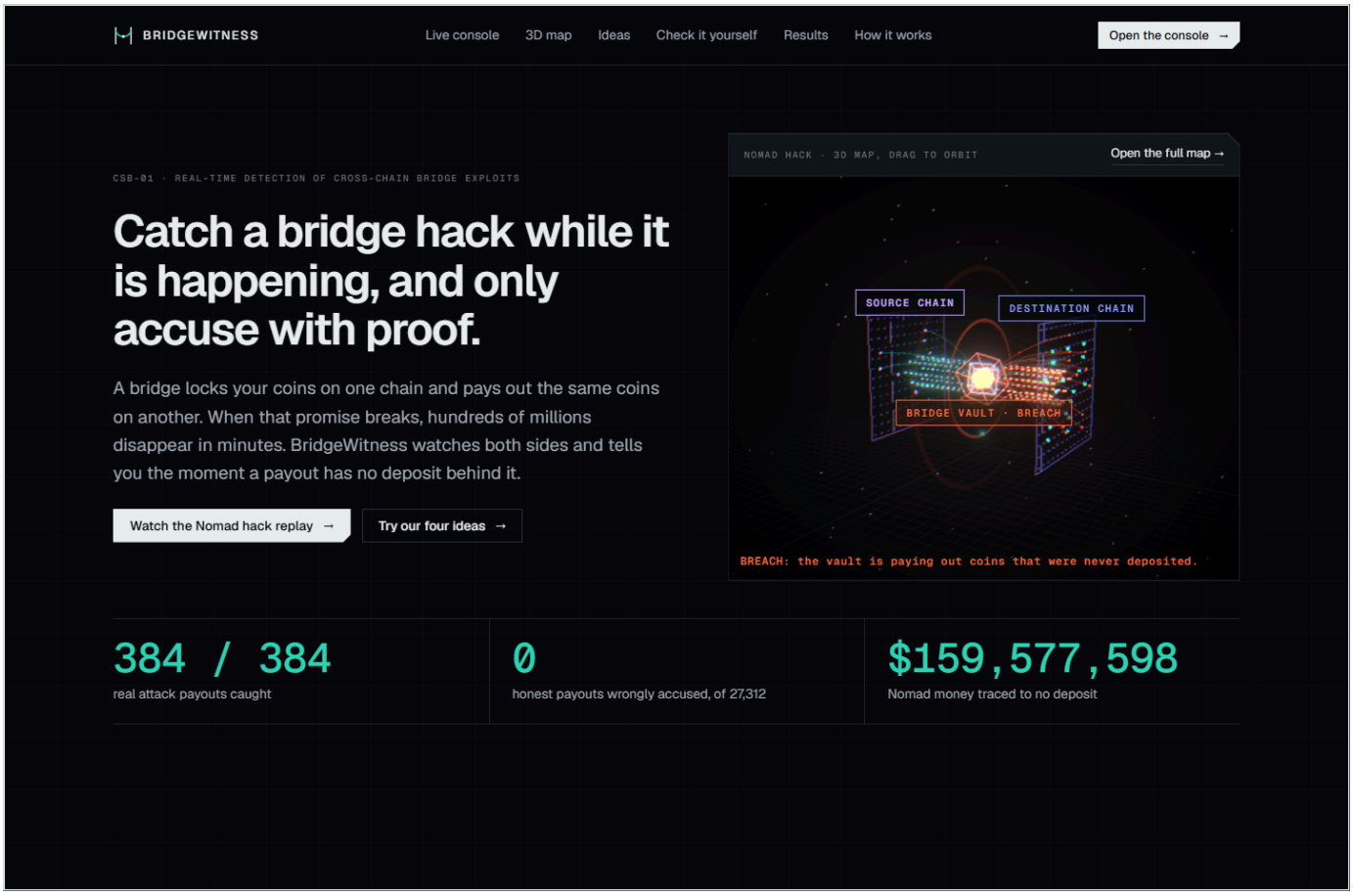
The challenge brief and where to see each part

The brief asks for	Where it is
A dashboard over a live or replayed feed	Live console, 3D map
Volume spikes	Idea 2 · Contagion Radar, charts in the Live console
Unusual signer behaviour	Idea 3 · Signer DNA
Mismatched deposit and payout pairs	Idea 4 · Ledger Pairing
A risk-scored alert with the suspected exploit type	Alert card (Live console), Idea 1 · Accusation Budget

PAGE · /

Landing page

The front door. It explains the problem in one screen and proves the result with three numbers.



What you can do here

1. Read the headline: catch a bridge hack while it is happening, and only accuse with proof.
2. Watch the 3D map on the right replay the real Nomad hack. Drag it to turn it around.
3. Read the three proof numbers: 384 of 384 real attack payouts caught, 0 honest payouts wrongly accused (out of 27,312), and $159.6M traced to no deposit.
4. Scroll to Key features, Why this matters, the four ideas and the five-step explanation. Every tile is a link to the page that shows it working.

Why it matters

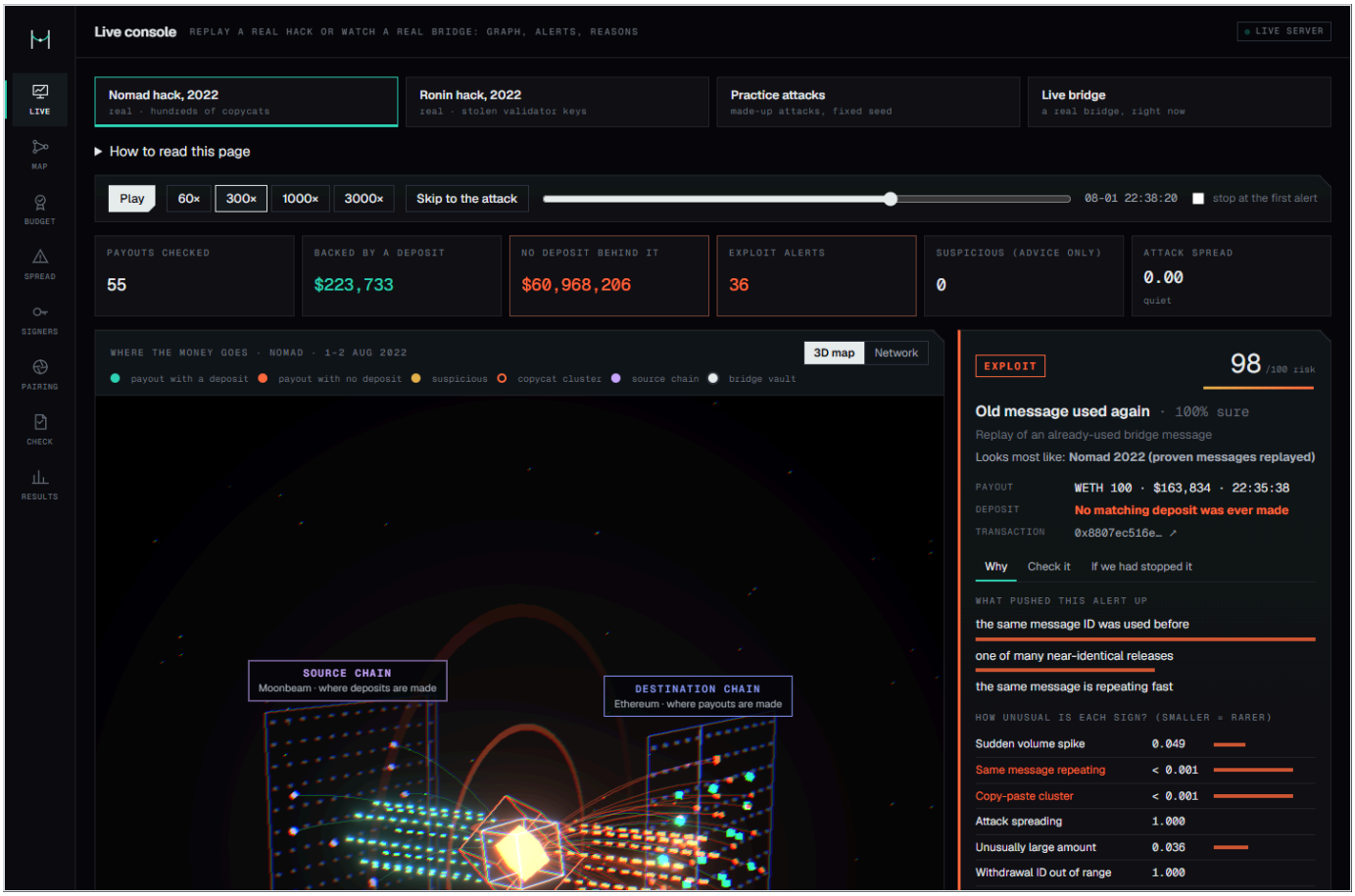
A judge sees the whole product in under a minute: what a bridge is, why hacks happen, what we built, and the evidence.

> The numbers come straight from the measured results file, not typed in by hand.

PAGE · /CONSOLE/LIVE

Live console

The main dashboard. It replays a real hack (or a made-up practice attack, or a real bridge right now) and shows alerts as they appear.



What you can do here

1. Pick a feed at the top: Nomad hack, Ronin hack, Practice attacks, or A real bridge right now.
2. Press Play. Every payout on the destination chain is checked against the deposits made on the source chain.
3. The replay stops at the first alert. The alert card shows a risk score from 0 to 100, the attack type, and the top reasons.
4. Use the 3D map / Network switch to see the money flow as a 3D scene or as a labelled diagram.
5. Read the three charts under the map: money in versus money out, is the attack spreading, and the alarm evidence.

Why it matters

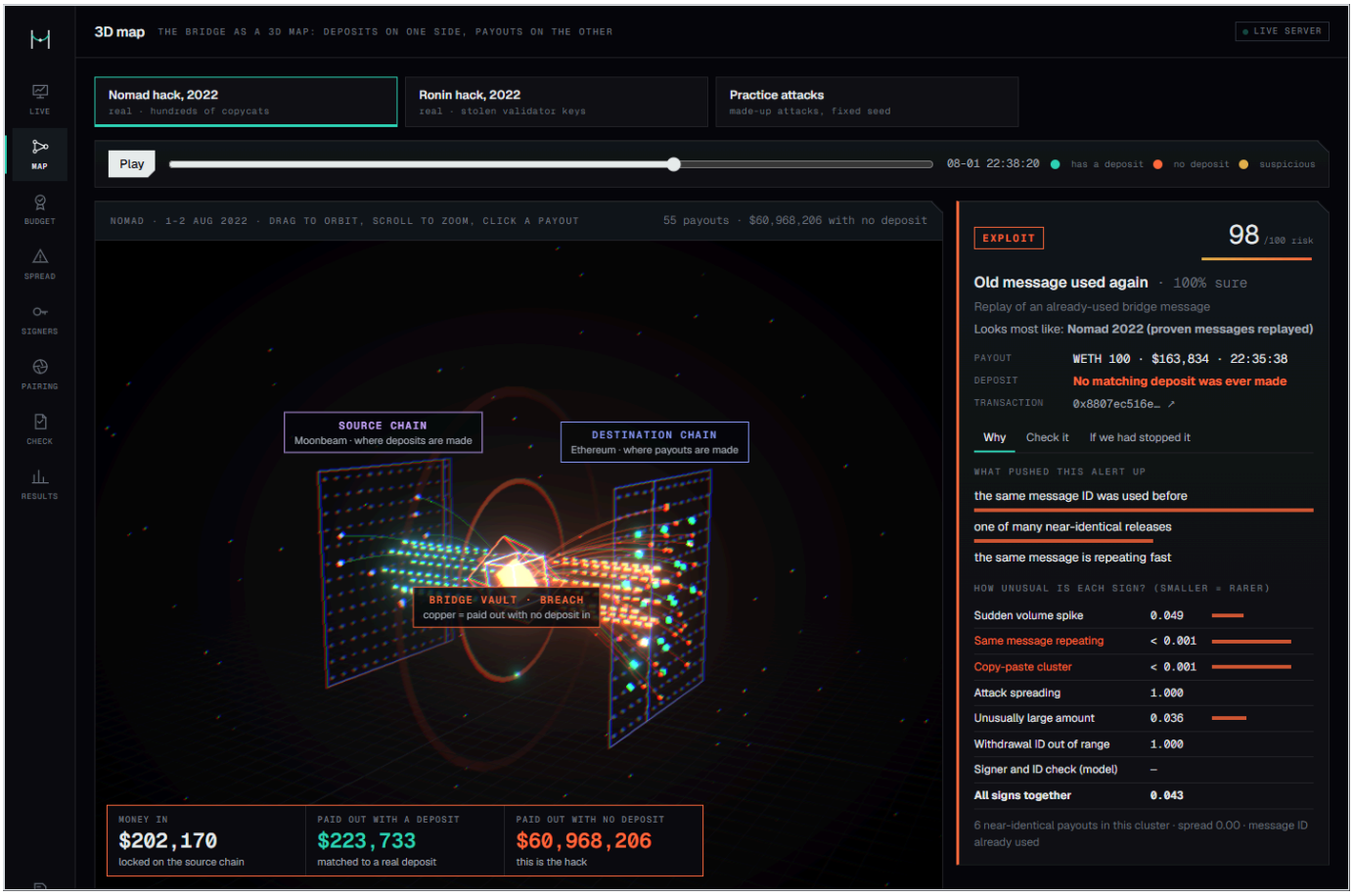
This answers the challenge brief directly: a dashboard over a live or replayed feed that flags volume spikes, unusual signers and mismatched pairs, with a risk-scored alert and the suspected exploit type.

> The 'real bridge right now' feed uses the public Across API and only runs the simple deposit rule. Our learned models say 'not enough history' there instead of guessing.

PAGE · /CONSOLE/MAP

3D map

The same replay as a 3D scene, built to be understood at a glance.



What you can do here

1. Left block of squares: the source chain, where deposits are made. Right block: the destination chain, where payouts are made.
2. The glowing cube in the middle is the bridge vault. Teal packets are money that has a deposit behind it.
3. When a payout has no deposit, the vault turns copper, flashes BREACH, and copper packets stream out to the right.
4. The three numbers at the bottom keep score: money in, paid out with a deposit, paid out with no deposit.
5. Click any dot on the right to open its alert card. Drag to orbit, scroll to zoom.

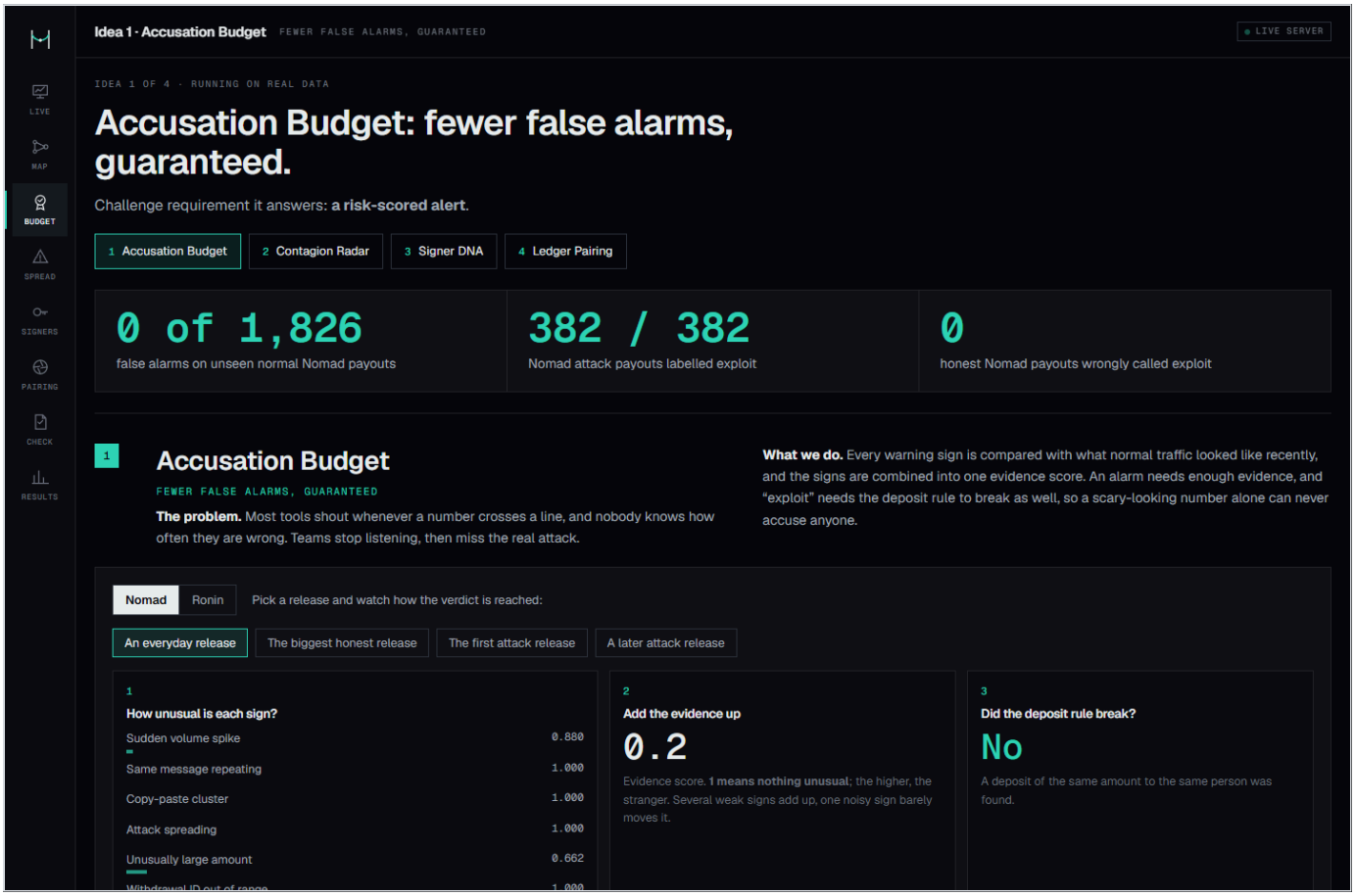
Why it matters

It turns an abstract idea (coins out must never exceed coins in) into something anyone can see. The no-deposit total jumping to $126.9M is the hack.

PAGE · /CONSOLE/BUDGET

Idea 1 · Accusation Budget

Our answer to false alarms. Many tools shout whenever one number looks big. We only accuse when the evidence adds up, and we measure how often we are wrong.



What you can do here

1. Choose Nomad or Ronin, then pick a payout (an everyday one, the biggest honest one, the first attack, a later attack).
2. Step 1 shows how unusual each warning sign is compared with recent normal traffic.
3. Step 2 adds the signs into one evidence score. Several weak signs add up; one noisy sign barely moves it.
4. Step 3 asks whether the deposit rule broke. Only a broken rule can produce the word EXPLOIT.

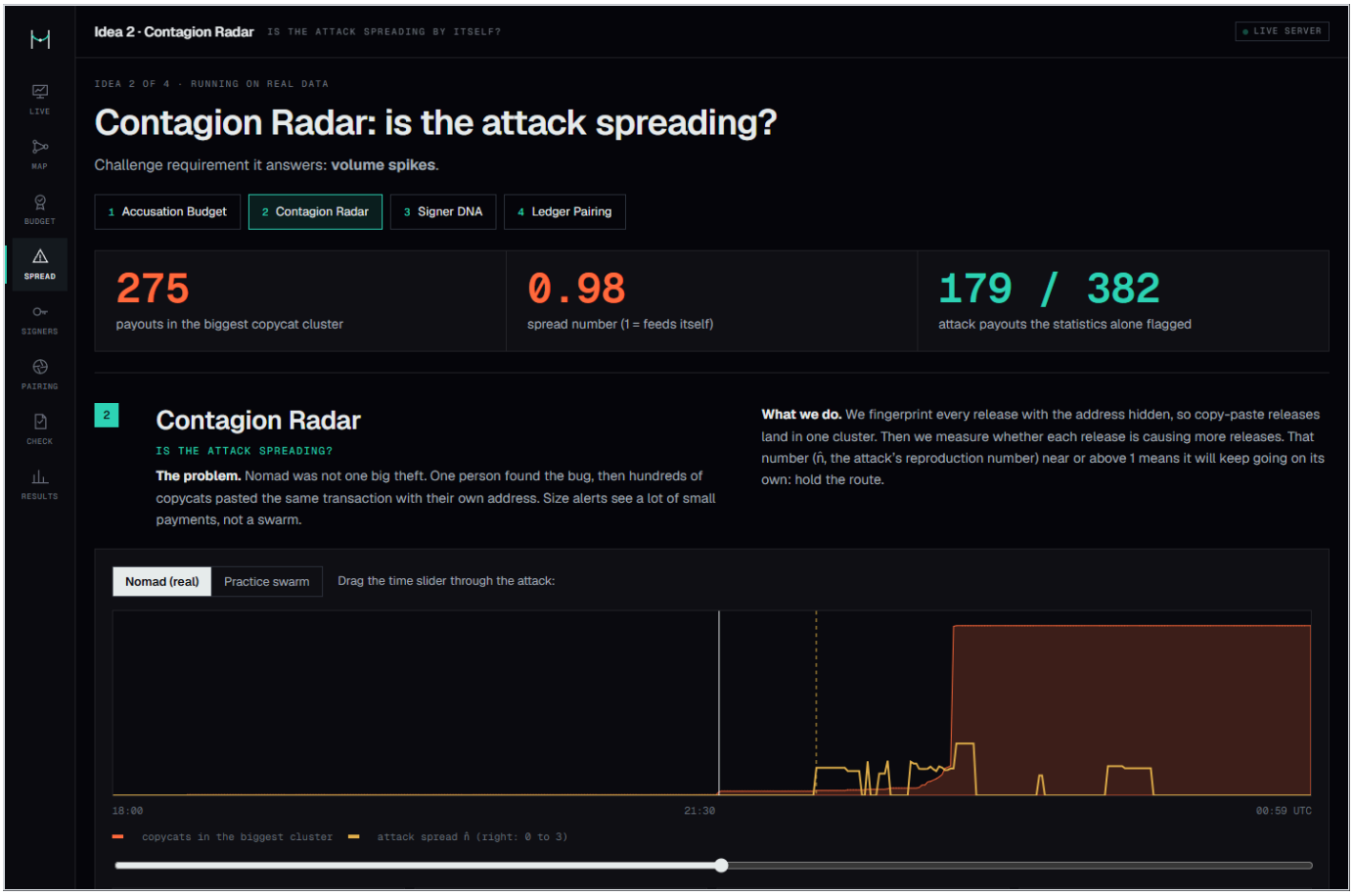
Why it matters

On days of normal traffic the model had never seen, there were 0 false alarms out of 1,826 payouts on Nomad. Statistics alone can only say 'suspicious'; the rule makes the final call.

PAGE · /CONSOLE/CONTAGION

Idea 2 · Contagion Radar

Detects whether an attack is spreading by itself, the way an outbreak does.



What you can do here

1. Drag the time slider through the Nomad hack and watch the chart.
2. The orange area is money released with no deposit. The yellow line is the spread number (n-hat).
3. Near or above 1 means each stolen payout is causing about one more, so the attack will keep going on its own.
4. Copy-paste payouts are grouped into one cluster by fingerprinting each payout with the address hidden.

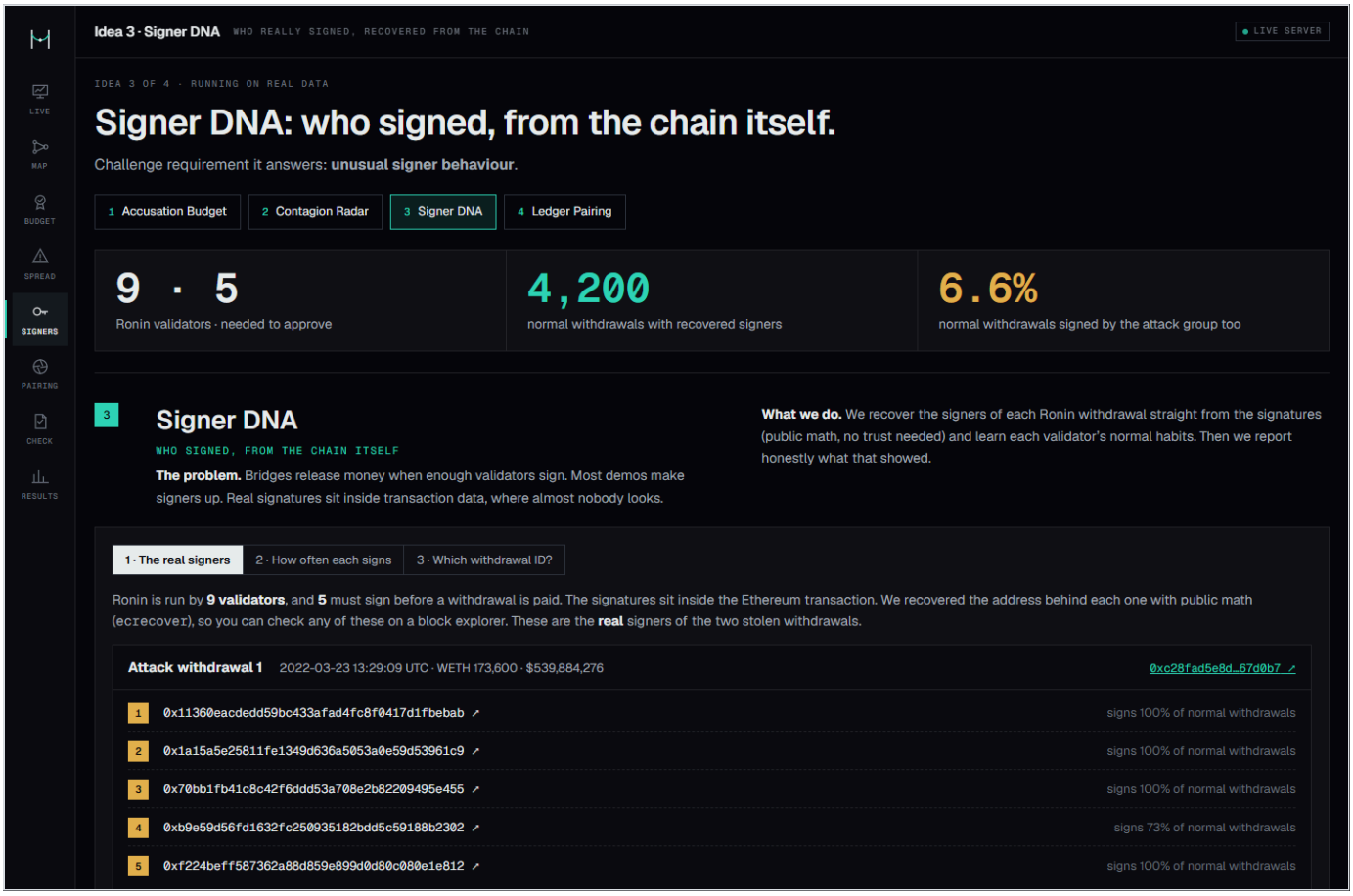
Why it matters

Nomad was not one theft. One person found a bug and hundreds of copycats pasted the same transaction. Size alerts see many small payments; we see a swarm. The biggest cluster reached 275 payouts.

PAGE · /CONSOLE/SIGNERS

Idea 3 · Signer DNA

Recovers who really signed each Ronin withdrawal, straight from the signatures on the blockchain.



What you can do here

1. Tab 1 lists the real addresses that signed the two stolen Ronin withdrawals, recovered from the signatures inside each transaction, with a link to check each on a block explorer.
2. Tab 2 shows how often each of the 9 validators signs normal withdrawals (4,200 recovered). Amber marks the ones that signed the attack.
3. Tab 3 shows the real signal: the withdrawal ID.

Why it matters

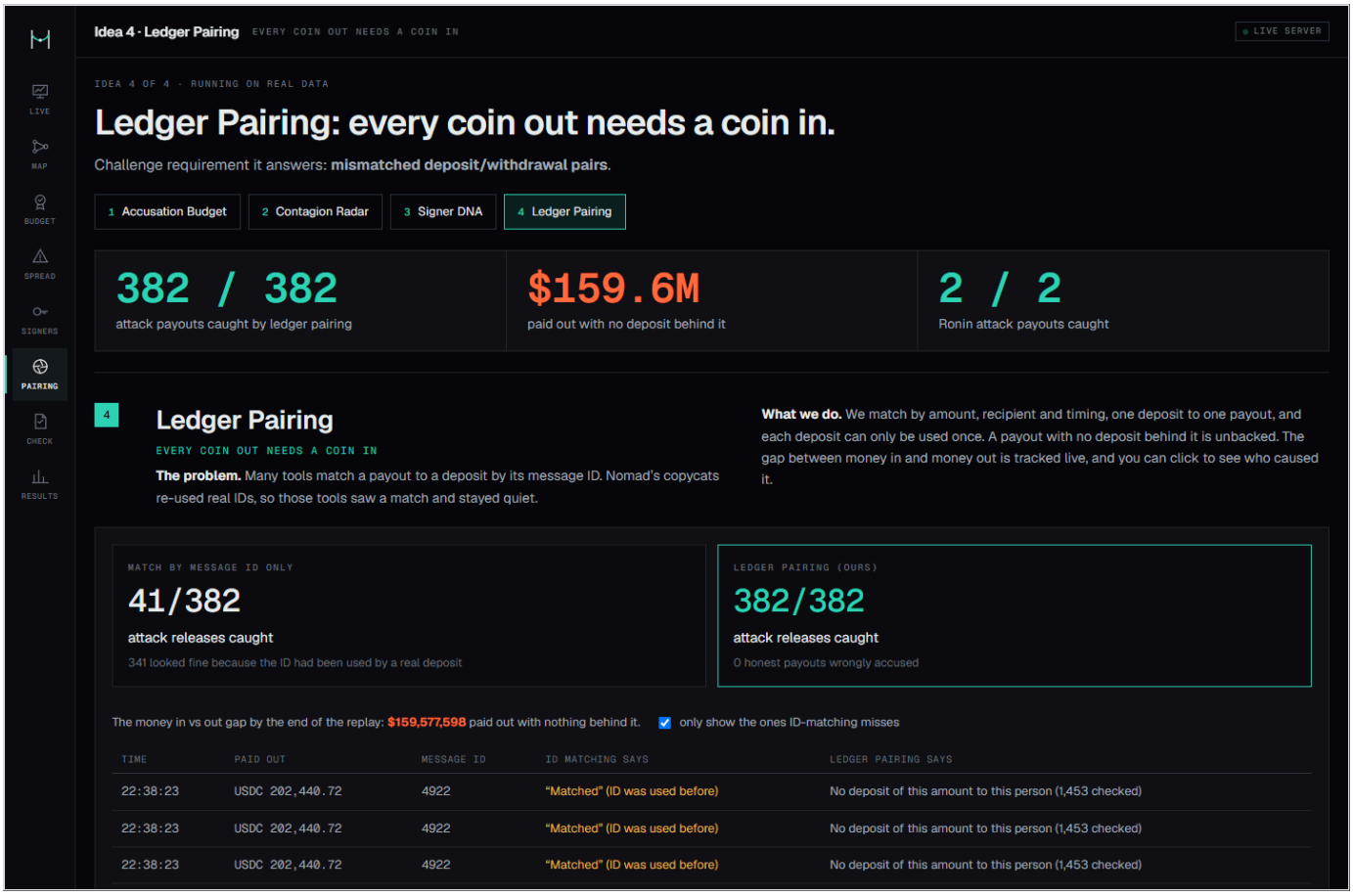
Honest finding: the attackers' signer group was ordinary (it also signed 6.6% of normal withdrawals), so who signed is not the clue. The clue is the withdrawal ID, 2,000,000 against a normal maximum of 1,064,238. We report this openly.

> Only Ronin has recoverable signatures in our data. For Nomad the signer model says 'not applicable' instead of making something up.

PAGE · /CONSOLE/PAIRING

Idea 4 · Ledger Pairing

Every coin paid out must match a coin locked. We pair each payout to one deposit and show the gap.



What you can do here

1. Compare the two boxes: matching by message ID only catches 41 of 382 attacks; our pairing catches 382 of 382.
2. Scroll the table to see each payout, what ID-matching says, and what pairing says.
3. Tick the filter to show only the payouts that ID-matching misses.

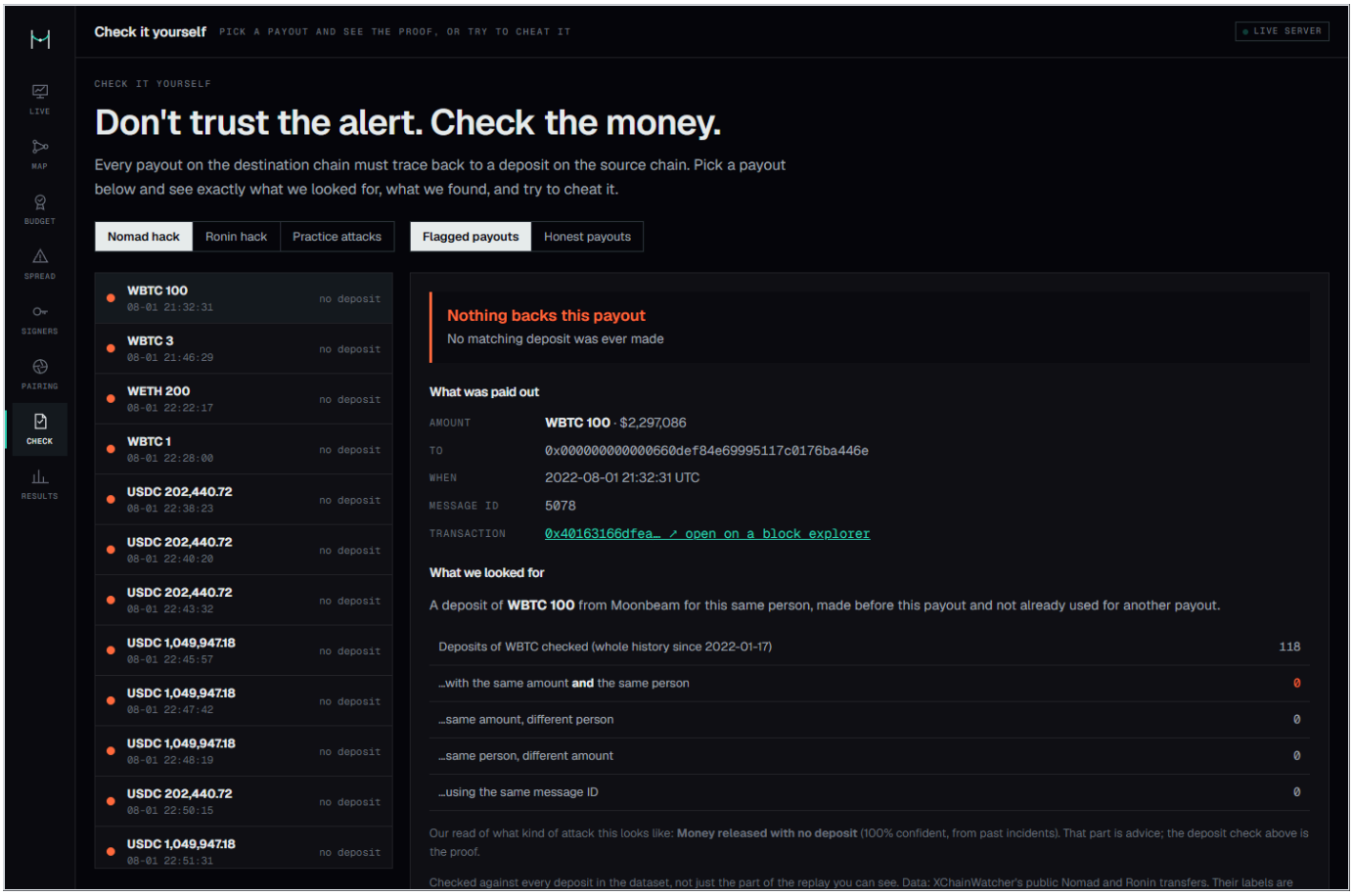
Why it matters

Nomad's copycats re-used real message IDs, so tools that match by ID saw nothing wrong. We match by amount, recipient and timing, and each deposit can be used only once.

PAGE · /CONSOLE/EVIDENCE

Check it yourself

Proof anyone can repeat. Pick any payout and see exactly what we searched.



What you can do here

1. Pick a payout from the list.
2. Read the search: how many deposits were checked and what we looked for (same amount and recipient, same message ID, and so on).
3. Try the cheat: change the amount or recipient and watch the verdict change.

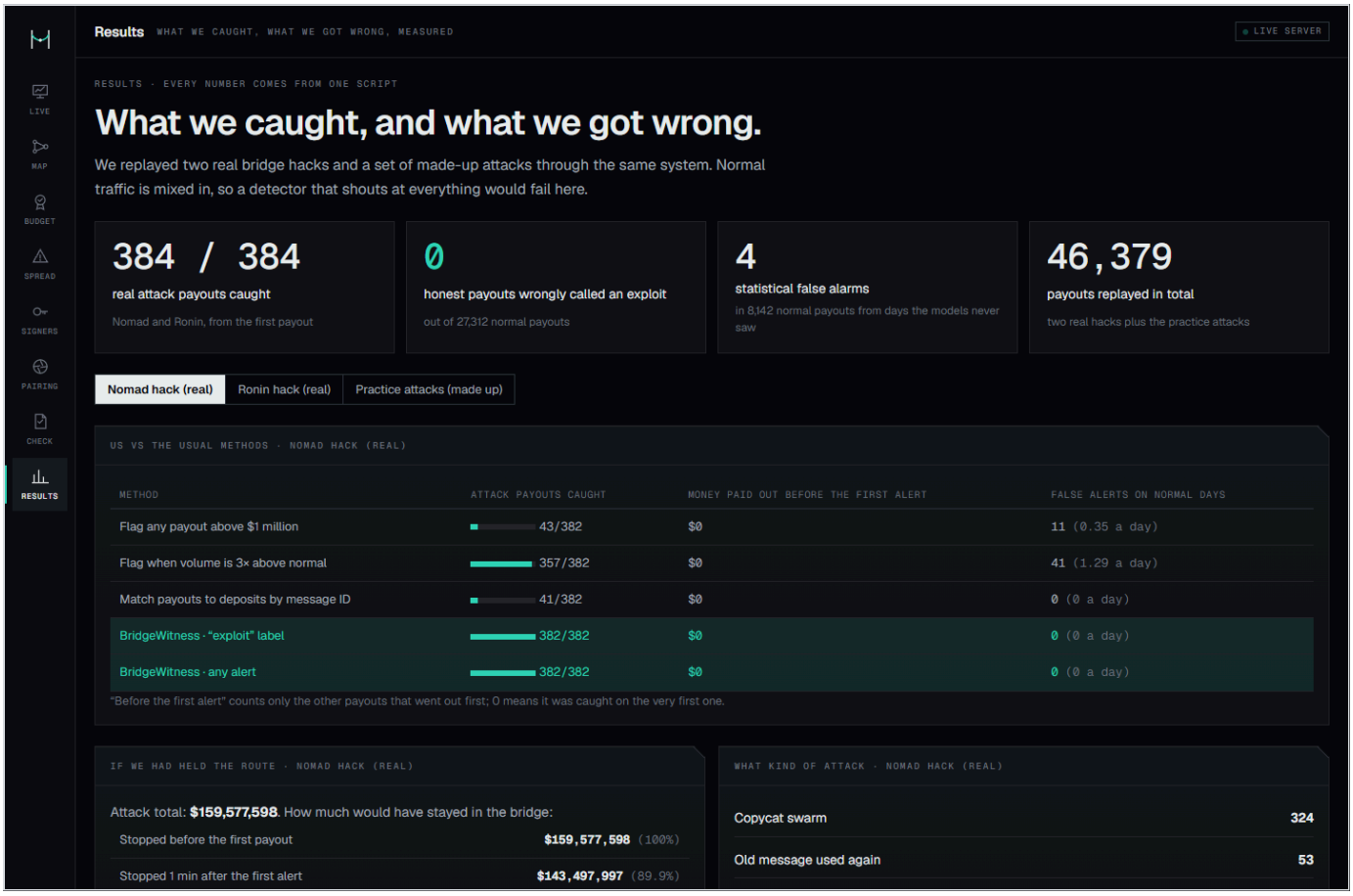
Why it matters

A security tool that says 'trust me' is not enough. Here a judge can test the verdict, not just read it.

PAGE · /CONSOLE/RESULTS

Results

Everything we measured, including the things we got wrong.



What you can do here

1. See attack payouts caught and honest payouts wrongly accused, per incident.
2. See the scorecard against simple baselines: whale threshold, volume z-score and message-ID matching.
3. See the what-if: holding funds before the first payout would have kept $159.6M of Nomad (100%).
4. Read the honest limits: where the models are weaker.

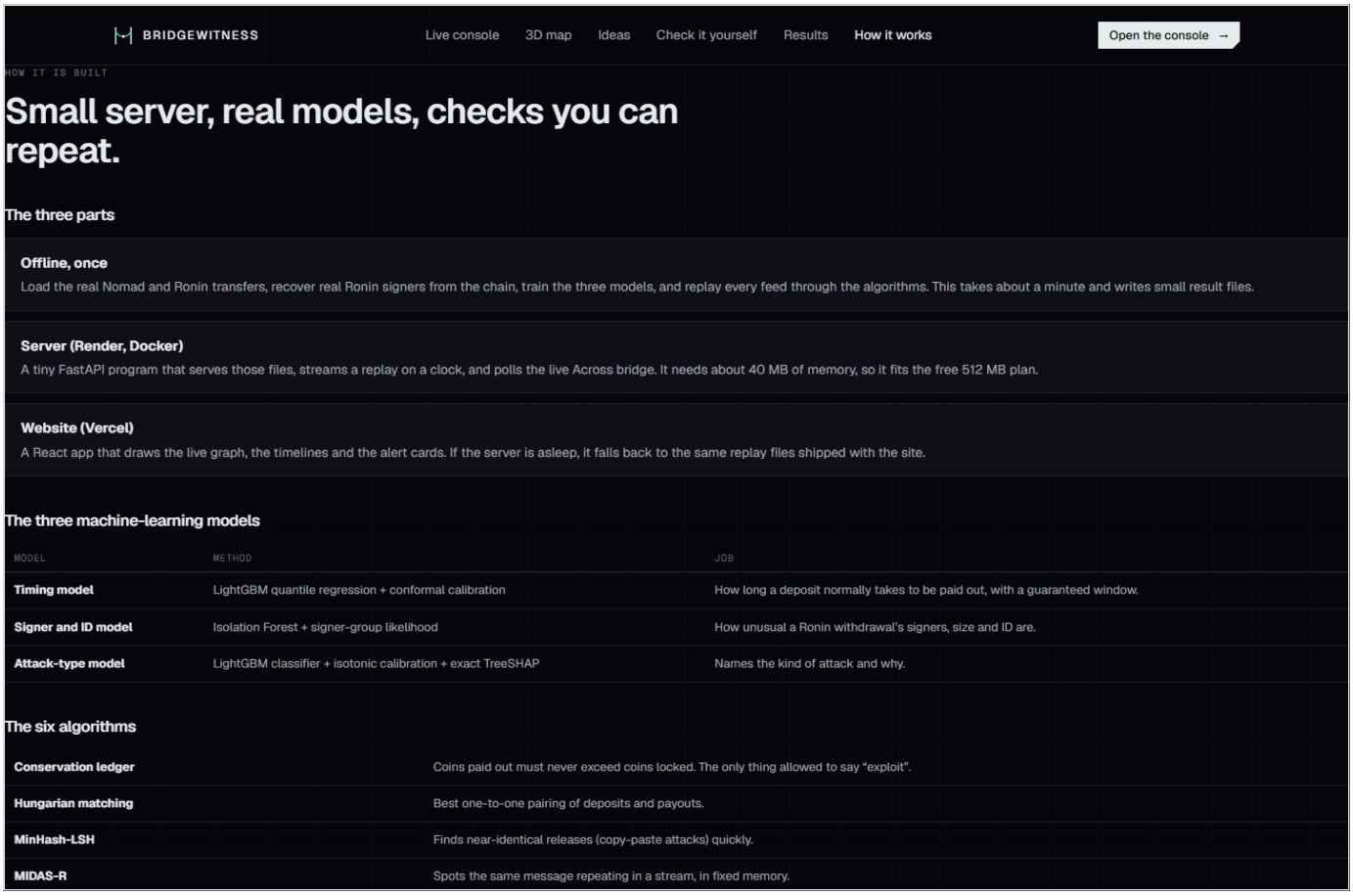
Why it matters

Judges trust numbers that come with their limits. The ML3 attack-type accuracy is on synthetic rows only, and real incidents are reported one by one.

PAGE · /DOCS/ARCHITECTURE

How it works

The pipeline in plain steps, for judges who want to look under the hood.



What you can do here

1. Read: deposits and payouts from the dataset or a live bridge.
2. Pair: match each payout to a deposit.
3. Measure: spread, volume, signers and size against normal days.
4. Decide: EXPLOIT only if the deposit rule breaks, otherwise at most SUSPICIOUS.
5. Explain: attack type, top reasons, closest past hack, and a repeatable check.

Why it matters

It lists the six algorithms and three models and what each is used for.

What is under the hood

Six algorithms

Name	What it does, simply
Conservation ledger + pairing	Matches each payout to one deposit; tracks money in versus money out.
Copy-paste detector (MinHash)	Groups payouts that are copies of each other, ignoring the address.
Repeat-message score (MIDAS-R)	Notices the same message showing up again and again in a burst.
Spread number (Hawkes)	Measures whether each payout triggers more payouts, like an outbreak.
Evidence stack (conformal + e-values + e-CUSUM)	Combines warning signs into one score with a stated false-alarm rate.
Volume spike check	Compares the last 10 minutes with the usual 7-day rate.

Three machine-learning models

Model	What it does	Trained on
Delay window	Predicts how long a payout normally takes after its deposit, with a 99% coverage target (98.8% measured on unseen data).	27,312 real matched pairs
Signer and ID anomaly	Spots unusual signer groups and withdrawal IDs. Ronin only.	4,200 real Ronin withdrawals with recovered signers
Attack type	Names the likely attack (copycat swarm, message replay, key compromise and so on) and shows the top reasons. It can never call something benign after the rule broke.	Synthetic scenarios built over real normal rows

Honest limits (we say these out loud)

1. Two real hacks plus practice attacks show the idea works. They are not a promise for every future attack.
2. The reference answers come from the XChainWatcher project's matching rules. We reproduce them; we did not discover the hacks independently.
3. The Ronin attackers' signer group looked ordinary. Who signed is not the clue; the withdrawal ID is.
4. The attack-type model's accuracy is on synthetic rows only. Real incidents are reported one by one.
5. The live bridge feed (Across) only runs the deposit rule; the models say "not enough history" there.
6. Dollar values use the dataset's historic prices. "Money kept" figures are estimates, not recovered funds.
7. The public Ronin loss estimate is $624M; the dataset total is $565.4M.

Data and credits

Nomad and Ronin transfers: XChainWatcher (Augusto et al., arXiv 2410.02029), MIT licence. Live feed: Across public API.

Tech, for the curious

Python backend (FastAPI, LightGBM, scikit-learn) packaged in Docker for Render. Website built with React, Vite and TypeScript, with three.js for the 3D map, deployed on Vercel. A self-check of 15 tests must pass before every release.